

UNIVERSIDADE DE SÃO PAULO

INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

PROJETO DE PESQUISA FAPESP - RELATÓRIO PARCIAL

O Problema de Visita de Polígonos

LINHA DE FOMENTO: BOLSA NO PAÍS - REGULAR - INICIAÇÃO CIENTÍFICA

Pesquisador Responsável:	Ernesto G. Birgin
Instituição Sede:	Universidade de São Paulo
Bolsista:	Gabriel F. Ushijima
Número do Processo:	2025/13861-1
Período de Vigência:	01/04/2026 a 31/03/2027
Período Coberto:	01/04/2026 a 01/06/2026

Bolsista:

Pesquisador Responsável:

Gabriel Freire Ushijima

Ernesto G. Birgin

São Paulo, SP
17 de junho de 2026

Resumo

Neste relatório parcial, descrevemos as atividades realizadas nos 2 primeiros meses da bolsa de Iniciação Científica, dedicadas ao estudo e implementação de algoritmos para o Problema de Visita de Polígonos (*Touring Polygons Problem*, TPP). O TPP consiste em determinar um caminho de comprimento mínimo na norma Euclidiana partindo de um ponto inicial s , visitando uma sequência ordenada de polígonos $P_1, \dots, P_k$ no plano e terminando em um ponto t . Em sua formulação geral, o problema é NP-difícil; por esse motivo, focamos o período coberto por este relatório na variante em que os polígonos são convexos e disjuntos, para a qual existe solução polinomial.

Estudamos o artigo de Dror et al. [2], que introduz o problema e apresenta um algoritmo exato baseado nas estruturas de região de primeiro contato e mapa de último passo, com complexidade $O(nk \log(n/k))$, onde n é o número total de vértices e k é o número de polígonos. Implementamos esse algoritmo em C++23, produzindo pseudocódigo detalhado, análise de complexidade e experimentos computacionais que confirmam o comportamento assintótico previsto. Desenvolvemos também uma ferramenta de visualização interativa, disponível em <https://yushipy.github.io/PersonalPage/Visualizer/index.html>, que permite inspecionar instâncias, soluções e mapas de último passo, e que estenderemos para suportar o caso não convexo nas próximas etapas do projeto.

Palavras-chave: problema de visita de polígonos, otimização geométrica, complexidade computacional, experimentos numéricos.

Sumário

1	Introdução	3
2	O Problema de Visita de Polígonos Convexos Disjuntos	3
2.1	Definições	3
2.2	O Algoritmo	6
2.2.1	Região de Primeiro Contato	6
2.2.2	Mapa de Último Passo	6
2.2.3	Localização de Pontos e Busca Binária	8
2.2.4	Consulta ao Caminho	9
2.2.5	Implementação Completa	10
2.3	Análise de Complexidade	11
2.4	Experimentos Computacionais	12
3	Ferramenta de Visualização	14
4	Plano de Atividades para o Próximo Período	15

1 Introdução

O Problema de Visita de Polígonos (*Touring Polygons Problem*, TPP), introduzido por Dror et al. [2], é um problema de otimização geométrica que consiste em determinar um caminho de comprimento mínimo na norma Euclidiana partindo de um ponto s , visitando uma sequência ordenada de polígonos $P_1, \dots, P_k$ no plano e terminando em um ponto t . O problema pode ser interpretado como um caso particular do Problema do Caixeiro Viajante com Regiões (*Traveling Salesman Problem with Neighborhoods*, TSPN) [1], no qual as regiões são polígonos e a ordem de visita é pré-determinada.

O TPP apresenta variantes de dificuldade bastante distinta. Quando os polígonos são convexos e disjuntos, o problema admite solução exata em tempo polinomial: Dror et al. [2] apresentam um algoritmo de complexidade $O(nk \log(n/k))$, onde n é o número total de vértices. Quando os polígonos podem ser não convexos, o problema torna-se NP-difícil, mesmo com a ordem de visita fixada [2].

Este relatório cobre os 2 primeiros meses da bolsa, dedicados ao estudo e implementação do caso convexo disjunto, que é o mais simples e serve de base para as etapas seguintes do projeto. Estudamos em detalhe o algoritmo de Dror et al. [2], baseado nas estruturas de região de primeiro contato e mapa de último passo. Implementamos o algoritmo em C++23 com busca binária para localização de pontos nos mapas de último passo, atingindo a complexidade $O(nk \log(n/k))$ prevista no artigo original. Realizamos experimentos computacionais para verificar o comportamento assintótico e comparar variantes da implementação. Desenvolvemos também uma ferramenta de visualização interativa que permite inspecionar instâncias, soluções e mapas de último passo.

O restante deste relatório está organizado da seguinte forma. A Seção 2 apresenta a definição formal do problema, as estruturas centrais do algoritmo e sua implementação, juntamente com a análise de complexidade e os experimentos computacionais. A Seção 3 descreve a ferramenta de visualização desenvolvida. Por fim, a Seção 4 descreve o plano de atividades para o próximo período.

2 O Problema de Visita de Polígonos Convexos Disjuntos

Consideramos o TPP na variante em que os polígonos são convexos e disjuntos. Utilizamos tanto a formulação quanto a solução apresentadas em [2] como base para a implementação desenvolvida. O problema é definido formalmente a seguir:

TPP Convexo Disjunto: Dados pontos $s, t \in \mathbb{R}^2$ e uma sequência de polígonos convexos disjuntos $P_1, \dots, P_k$, determinar pontos $p_i \in P_i$ para $i = 1, 2, \dots, k$ de forma que o caminho que parte de s , passa por $p_1, \dots, p_k$ e termina em t possua comprimento mínimo na norma Euclidiana.

2.1 Definições

As definições e propriedades a seguir são baseadas em [2]. Supõe-se que um polígono é representado por uma sequência de vértices em sentido anti-horário, e que $P_0 = \{s\}$ e $P_{k+1} = \{t\}$.

Utilizamos o operador $\times$ para denotar o produto vetorial em $\mathbb{R}^2$: para dois vetores $a = (a_x, a_y)$ e $b = (b_x, b_y)$, define-se $a \times b = a_x b_y - a_y b_x$. Note que $a \times b > 0$ se b está à esquerda de a , e $a \times b < 0$ se b está à direita de a .

Caminho Parcial Ótimo (i -path): Um i -path ($0 \leq i \leq k$) até um ponto $p \in \mathbb{R}^2$ é um caminho de comprimento mínimo que se inicia em s , termina em p e visita os polígonos $P_1, P_2, \dots, P_i$ nessa ordem. Como se trabalha com polígonos convexos, o i -path até qualquer ponto é único, o que ficará evidente a partir das propriedades dos mapas de último passo. O 0-path até p é simplesmente o segmento $\overline{sp}$, e o k -path até t corresponde à solução do problema. A Figura 1 ilustra um exemplo de 2-path até t .

Região de Primeiro Contato (T_i): A região de primeiro contato T_i ($1 \leq i \leq k$) de um polígono P_i consiste nos pontos p pertencentes à fronteira de P_i tais que o último segmento do $(i-1)$ -path até p não atravessa o interior de P_i . Como P_i é convexo, T_i é sempre composta por uma região contígua de arestas de P_i . A Figura 2 ilustra as regiões T_1 e T_2 .

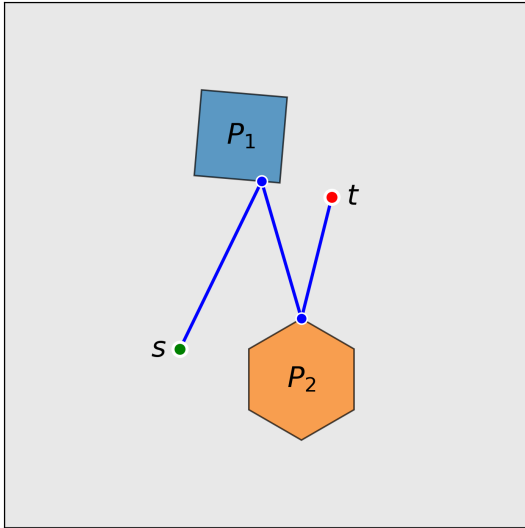


Figura 1: Caminho mínimo de s até t visitando P_1 e P_2 .

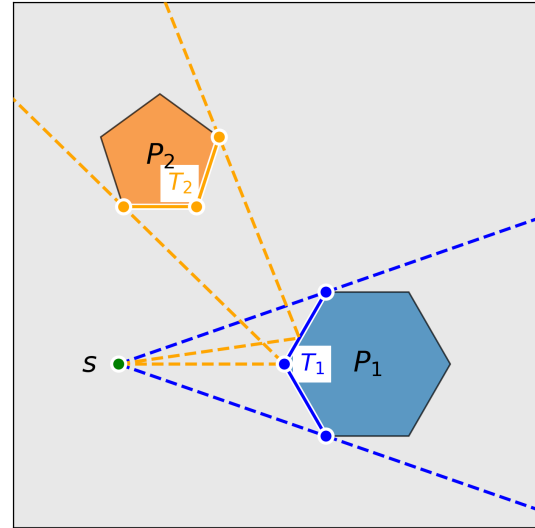


Figura 2: Região de primeiro contato T_1 de P_1 e T_2 de P_2 .

Mapa de Último Passo (S_i): O mapa de último passo S_i ($1 \leq i \leq k$) de um polígono P_i é uma partição do plano em regiões associadas a cada vértice e a cada aresta de P_i . Um ponto $p \in \mathbb{R}^2$ pertence à região associada a um vértice v se o i -path até p possui $\overline{vp}$ como último segmento; pertence à região associada a uma aresta e se o último segmento do i -path até p parte do interior de e ; e pertence à região de atravessar caso contrário, ou seja, quando o i -path até p coincide com o $(i-1)$ -path até p . Apenas vértices e arestas em T_i possuem regiões não degeneradas. A Figura 3 apresenta um exemplo de mapa de último passo.

Dado um ponto $p \in \mathbb{R}^2$ e o mapa S_i , é possível determinar o i -path até p eficientemente a partir do $(i-1)$ -path, considerando três casos:

Região de Vértice: Se p pertence à região de um vértice v , o último segmento do i -path até p é $\overline{vp}$, e o i -path é simplesmente o $(i-1)$ -path até v seguido de $\overline{vp}$. A Figura 4 ilustra essa

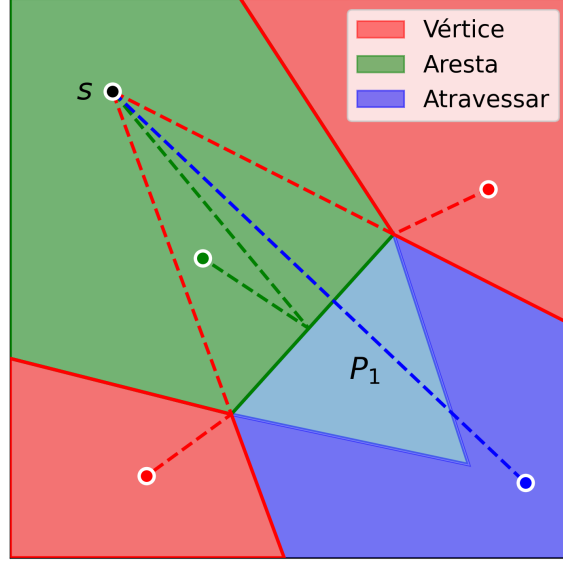


Figura 3: Exemplo do mapa de último passo S_1 de P_1 . Regiões de vértice em vermelho, de aresta em verde e de atravessar em azul, com o 1-path até um ponto representativo de cada região.

situação com dois pontos p^1 e p^2 em regiões de vértice distintas de P_2 . Note que S_2 informa apenas o último segmento do caminho, não o caminho completo: o último segmento até q^1 parte de um vértice de P_1 , enquanto o até q^2 parte do interior de uma aresta de P_1 , o que só se descobre consultando S_1 .

Região de Atravessar: Se p pertence à região de atravessar, o i -path até p coincide com o $(i-1)$ -path até p , pois o caminho ótimo simplesmente atravessa P_i sem desviar. A Figura 5 ilustra esse caso.

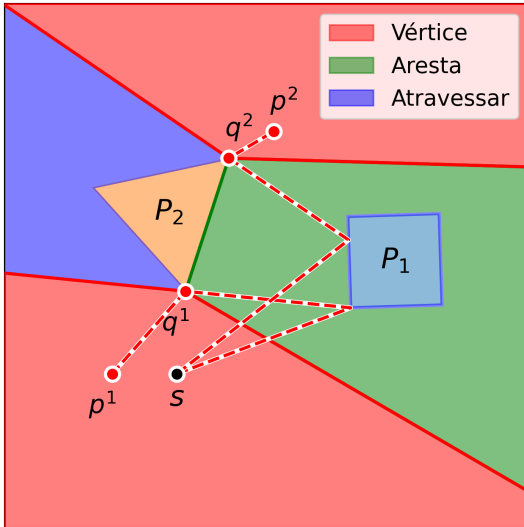


Figura 4: 2-path até p^1 e p^2 , em regiões de vértice de P_2 .

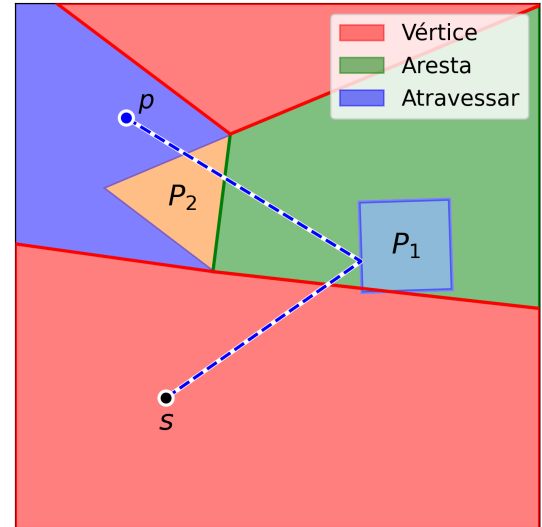


Figura 5: 2-path até p , em região de atravessar de P_2 .

Região de Aresta: Se p pertence à região de uma aresta $e = \overline{uv}$, reflete-se p em relação à reta que contém e , obtendo p' . O i -path até p é então a parte do $(i-1)$ -path até p' desde s até o

ponto q de interseção do último segmento desse caminho com e , seguida de $\overline{qp}$. A Figura 6 ilustra essa construção. As fórmulas para p' e q são:

$$p' = -p + 2u + 2 \frac{(p-u) \cdot (v-u)}{\|v-u\|^2} (v-u), \quad (1)$$

$$q = q' + \frac{(u-q') \times (v-u)}{(p'-q') \times (v-u)} (p'-q'). \quad (2)$$

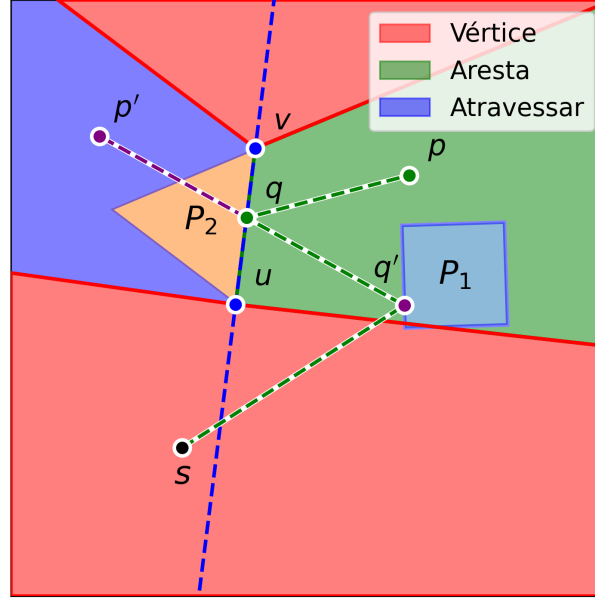


Figura 6: 2-path até p , em região de aresta de P_2 . O ponto p' é a reflexão de p , q' precede p' no 1-path, e q é a interseção de $\overline{q'p'}$ com e .

2.2 O Algoritmo

O algoritmo opera em duas fases. Na primeira, para cada $i = 1, \dots, k$, calcula-se T_i e em seguida S_i , usando os mapas já computados. Na segunda, consulta-se o k -path até t . A seguir, detalhamos cada etapa.

2.2.1 Região de Primeiro Contato

Para determinar T_i , basta verificar cada aresta de P_i : uma aresta $e = \overline{uv}$ pertence a T_i se e somente se $(v-u) \times (q-u) < 0$, onde q é o ponto que precede u no $(i-1)$ -path até u . Essa verificação é implementada no Algoritmo 1. Os algoritmos a seguir fazem uso de uma função auxiliar `ConsultaÚltimoPasso`($p, i, \dots$), que dado um ponto $p \in \mathbb{R}^2$ e um índice i , retorna o ponto que precede p no i -path até p ; essa função será implementada na Seção 2.2.4, após a definição de `LocalizaPonto`.

2.2.2 Mapa de Último Passo

Para calcular S_i , utilizam-se as condições de otimalidade local de [2]. Para cada vértice v de P_i com arestas incidentes e^1 e e^2 , calcula-se um par de raios (r^1, r^2) que delimita a região de

Algoritmo 1: Região de Primeiro Contato

Entrada: $i, s, (P_1, \dots, P_i), (T_1, \dots, T_{i-1}), (S_1, \dots, S_{i-1})$ **Saída** : Região de primeiro contato T_i de P_i como conjunto de arestas.

```
1 define RegiãoDePrimeiroContato( $i, s, \dots$ ):  
2    $T_i \leftarrow \emptyset$   
3   para cada aresta  $e = \overline{uv}$  de  $P_i$  :  
4      $q \leftarrow \text{ConsultaÚltimoPasso}(u, i - 1, \dots)$   
5     se  $(v - u) \times (q - u) < 0$  então:  
6        $T_i \leftarrow T_i \cup \{e\}$   
7   retorna  $T_i$ 
```

vértice associada a v em S_i , conforme o Lema 2.1.

Lema 2.1 *Seja v um vértice de P_i com arestas incidentes e^1 e e^2 , e seja q o ponto que precede v no $(i - 1)$ -path até v . O raio associado a uma aresta e incidente a v é $v - q$ se q está do lado interno de e , ou $r_\perp(q, v, e)$ caso contrário, onde $r_\perp(q, v, e)$ denota a reflexão do vetor $q - v$ em relação à reta perpendicular a e que passa por v . O par ordenado (r^1, r^2) assim obtido delimita exatamente a região de vértice de v em S_i .*

A reflexão $r_\perp(q, v, e)$ de $q - v$ em relação à perpendicular a $e = \overline{uv}$ em v é dada por:

$$r_\perp(q, v, e) = (q - v) - 2 \frac{(q - v) \cdot \hat{e}}{\|\hat{e}\|^2} \hat{e}, \quad \hat{e} = v - u,$$

onde $\cdot$ denota o produto interno em $\mathbb{R}^2$. Esse cálculo é implementado no Algoritmo 2.

Algoritmo 2: Mapa de Último Passo

Entrada: $i, s, (P_1, \dots, P_i), (T_1, \dots, T_i), (S_1, \dots, S_{i-1})$ **Saída** : Mapa de último passo S_i como associação de pares de raios aos vértices de P_i .

```
1 define Reflexão( $q, v, e$ ):  
2   se  $e \in T_i$  então:  
3     retorna reflexão de  $q - v$  em relação à perpendicular a  $e$  em  $v$   
4   senão  
5     retorna  $v - q$   
6 define MapaDeÚltimoPasso( $i, s, \dots$ ):  
7    $S_i \leftarrow \emptyset$   
8   para cada vértice  $v$  de  $P_i$  com arestas incidentes  $e^1$  e  $e^2$  :  
9      $q \leftarrow \text{ConsultaÚltimoPasso}(v, i - 1, \dots)$   
10     $r^1 \leftarrow \text{Reflexão}(q, v, e^1)$   
11     $r^2 \leftarrow \text{Reflexão}(q, v, e^2)$   
12    Associa  $(r^1, r^2)$  a  $v$  em  $S_i$   
13  retorna  $S_i$ 
```

2.2.3 Localização de Pontos e Busca Binária

Com S_i calculado, o passo central é localizar um ponto $p \in \mathbb{R}^2$ no mapa S_i . Descrevemos a seguir como realizar cada verificação em tempo $O(1)$.

Região de vértice. Seja v um vértice com raios associados (r^1, r^2) em S_i . O ponto p pertence à região de v se está no cone definido por v e pelos raios r^1 e r^2 em sentido anti-horário. Se o ângulo de abertura é inferior a 180° ($r^1 \times r^2 \geq 0$), exige-se $r^1 \times (p - v) > 0$ e $r^2 \times (p - v) < 0$; caso contrário ($r^1 \times r^2 < 0$), basta uma das duas condições.

Região de aresta. Seja $e = \overline{uv}$ uma aresta de T_i , com r^1 o segundo raio de u e r^2 o primeiro raio de v em S_i . O ponto p pertence à região de e se e somente se:

$$r^1 \times (p - u) > 0 \quad \text{e} \quad r^2 \times (p - v) < 0 \quad \text{e} \quad (v - u) \times (p - u) < 0.$$

Busca binária via arestas fictícias. Uma abordagem direta para localizar p em S_i verificaria cada região de vértice e aresta de T_i sequencialmente, com custo $O(|T_i|)$. Para melhorar isso, observa-se que as verificações acima se generalizam naturalmente para subconjuntos contíguos de vértices. Dado um subconjunto $v^l, \dots, v^r$ de vértices consecutivos de P_i , define-se uma *aresta fictícia* entre v^l e v^r usando os primeiros raios de v^l e v^r em S_i . A região associada a essa aresta fictícia contempla exatamente as regiões de vértice de v^l até v^{r-1} e as regiões de aresta entre eles, totalizando $r - l$ regiões de cada tipo. Formalmente, um ponto p pertence à região da aresta fictícia se e somente se pertence a alguma dessas regiões, ou a parte do interior de P_i delimitada por v^l e v^r . Como os polígonos são disjuntos, p nunca está no interior de P_i , de modo que essa ambiguidade não afeta a corretude do algoritmo. A verificação pode ser feita em tempo $O(1)$, considerando quatro casos dependendo de se os raios r^1 e r^2 estão do lado interno ou externo da aresta fictícia. A Figura 7 ilustra a construção e a Figura 8 os quatro casos.

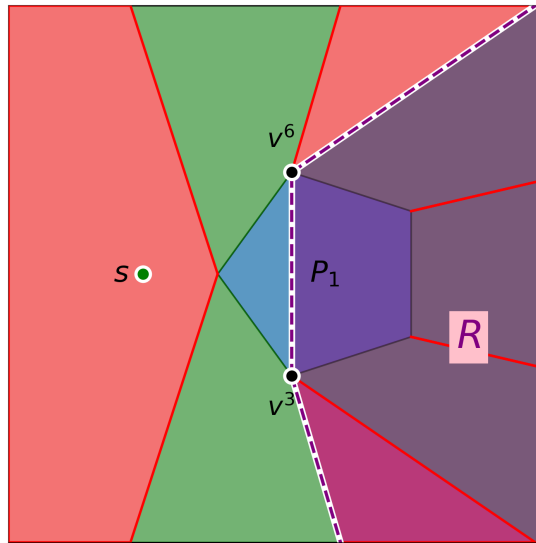


Figura 7: Aresta fictícia entre v^3 e v^6 de P_1 , cobrindo as regiões de vértice de v^3, v^4, v^5 e das arestas entre eles.

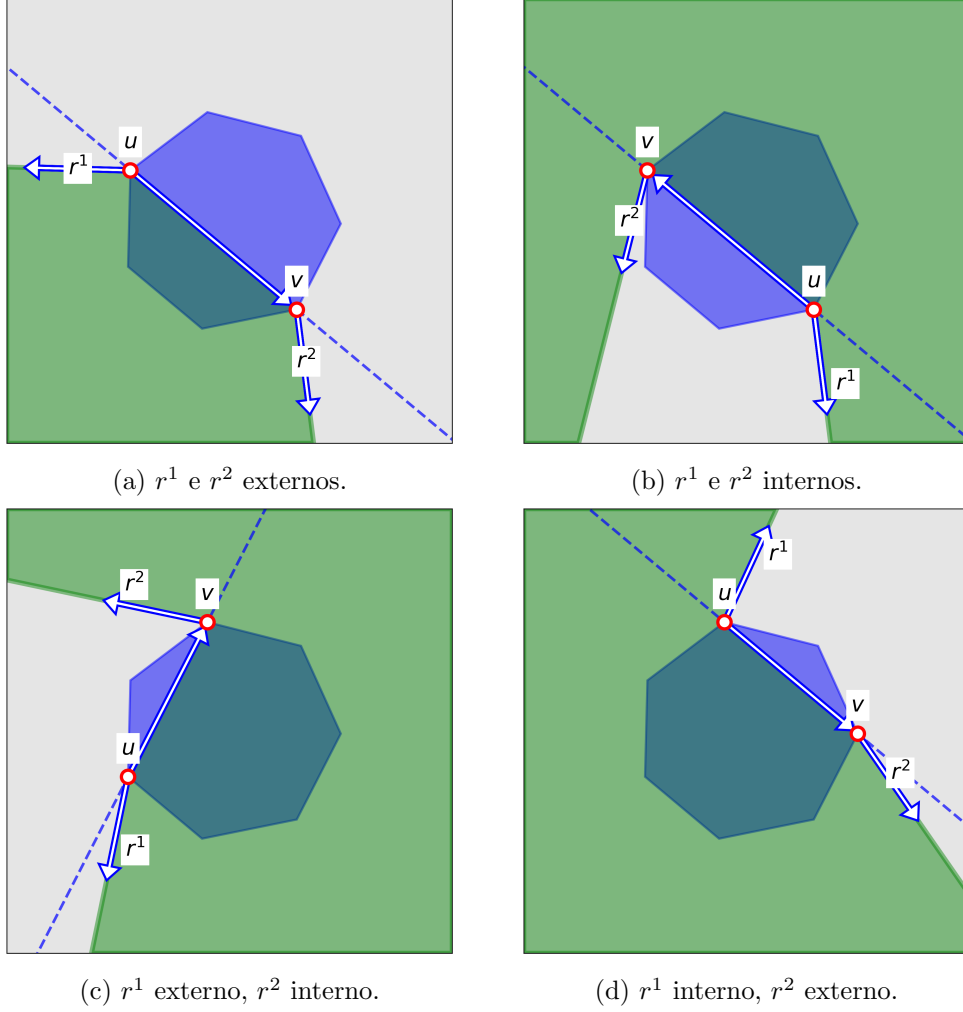


Figura 8: Casos para verificação de pertinência à região de uma aresta fictícia.

Com essa ferramenta, aplica-se busca binária sobre os vértices de P_i : mantém-se um intervalo $[l, r]$ com $l = 1$ e $r = |P_i| + 1$, e a cada passo verifica-se se p está entre v^l e v^{mid} . Quando $l + 1 = r$, sabe-se que p está na região de v^l ou de $\overline{v^l v^r}$, e basta checar qual dos dois casos se aplica, retornando a região de atravessar se nenhum deles pertencer a T_i . O custo total é $O(\log |P_i|)$, implementado no Algoritmo 3.

2.2.4 Consulta ao Caminho

Com `LocalizaPonto` definida, podemos implementar as funções `ConsultaÚltimoPasso` e `ConsultaCaminho`. Ambas operam recursivamente sobre os mesmos três casos: se p está em uma região de vértice v , o i -path até p é o $(i - 1)$ -path até v seguido de $\overline{vp}$; se está em uma região de aresta $e = \overline{uv}$, reflete-se p em relação à reta que contém e obtendo p' , calcula-se o $(i - 1)$ -path até p' e toma-se a interseção do último segmento com e como ponto de contato; se está na região de atravessar, o i -path até p coincide com o $(i - 1)$ -path até p .

A distinção entre as duas funções é que `ConsultaÚltimoPasso` retorna apenas o ponto que precede p no i -path, sem reconstruir o caminho completo. Isso permite uma otimização importante:

Algoritmo 3: Localiza Ponto

Entrada: p, P_i, T_i, S_i **Saída** : Região de S_i que contém p .

```
1 define VerificaAresta( $u, v, p, S_i$ ):
2    $r^1 \leftarrow$  primeiro raio de  $u$  em  $S_i$ 
3    $r^2 \leftarrow$  primeiro raio de  $v$  em  $S_i$ 
4    $c_1 \leftarrow r^1 \times (p - u) \geq 0$ 
5    $c_2 \leftarrow r^2 \times (p - v) \leq 0$ 
6    $c_3 \leftarrow (v - u) \times (p - u) \leq 0$ 
7   se  $(v - u) \times r^1 < 0$  então:
8     se  $(v - u) \times r^2 < 0$  então:
9       retorna  $c_1 \wedge c_2 \wedge c_3$ 
10    senão
11      retorna  $(c_1 \wedge c_3) \vee (c_2 \wedge \neg c_3)$ 
12  senão
13    se  $(v - u) \times r^2 < 0$  então:
14      retorna  $(c_1 \wedge \neg c_3) \vee (c_2 \wedge c_3)$ 
15    senão
16      retorna  $c_1 \vee c_2 \vee c_3$ 
17 define LocalizaPonto( $p, P_i, T_i, S_i$ ):
18    $l \leftarrow 1, \quad r \leftarrow |P_i| + 1$ 
19   enquanto  $l + 1 \neq r$  faça:
20      $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
21     se VerificaAresta( $v^l, v^{mid}, p, S_i$ ) então:
22        $r \leftarrow mid$ 
23     senão
24        $l \leftarrow mid$ 
25   se  $v^l \in T_i$  e  $p$  está na região de  $v^l$  então:
26     retorna região de  $v^l$ 
27   senão se  $\overline{v^l v^r} \in T_i$  então:
28     retorna região de  $\overline{v^l v^r}$ 
29   senão
30     retorna região de atravessar
```

no caso de região de vértice, v é imediatamente o ponto predecessor, sem necessidade de recursão adicional. Essa função é usada pelos Algoritmos 1 e 2, que precisam apenas do último passo e não do caminho completo. Ambas as funções estão implementadas no Algoritmo 4.

2.2.5 Implementação Completa

O algoritmo completo para o TPP Convexo itera sobre os polígonos calculando T_i e S_i em ordem, e ao final consulta o k -path até t , conforme o Algoritmo 5.

Algoritmo 4: Consulta Caminho e Último Passo

Entrada: $p, i, s, (P_1, \dots, P_i), (T_1, \dots, T_i), (S_1, \dots, S_i)$ **Saída** : i -path até p (ou apenas o ponto que o precede).

```
1 define ConsultaÚltimoPasso( $p, i, \dots$ ):
2   se  $i = 0$  então:
3     retorna  $s$ 
4    $R \leftarrow \text{LocalizaPonto}(p, P_i, T_i, S_i)$ 
5   se  $R$  é região de vértice  $v$  então:
6     retorna  $v$ 
7   senão se  $R$  é região de aresta  $e = \overline{uv}$  então:
8      $p' \leftarrow$  reflexão de  $p$  em relação à reta de  $e$ 
9      $q' \leftarrow \text{ConsultaÚltimoPasso}(p', i - 1, \dots)$ 
10    retorna interseção de  $\overline{q'p'}$  com  $e$ 
11  senão
12    retorna ConsultaÚltimoPasso( $p, i - 1, \dots$ )
13 define ConsultaCaminho( $p, i, \dots$ ):
14   se  $i = 0$  então:
15     retorna  $[s]$ 
16    $R \leftarrow \text{LocalizaPonto}(p, P_i, T_i, S_i)$ 
17   se  $R$  é região de vértice  $v$  então:
18     retorna ConsultaCaminho( $v, i - 1, \dots$ )  $\oplus [v]$ 
19   senão se  $R$  é região de aresta  $e = \overline{uv}$  então:
20      $p' \leftarrow$  reflexão de  $p$  em relação à reta de  $e$ 
21      $\mathbb{P} \leftarrow \text{ConsultaCaminho}(p', i - 1, \dots)$ 
22      $q \leftarrow$  interseção de  $\overline{(\text{último de } \mathbb{P})p'}$  com  $e$ 
23     retorna  $\mathbb{P} \oplus [q]$ 
24   senão
25     retorna ConsultaCaminho( $p, i - 1, \dots$ )
```

Algoritmo 5: TPP Convexo

Entrada: Pontos s e t , polígonos convexos disjuntos $(P_1, \dots, P_k)$ **Saída** : k -path até t .

```
1 define TPPConvexo( $s, t, P_1, \dots, P_k$ ):
2   para cada  $i \leftarrow 1$  até  $k$  :
3      $T_i \leftarrow \text{RegiãoDePrimeiroContato}(i, s, \dots)$ 
4      $S_i \leftarrow \text{MapaDeÚltimoPasso}(i, s, \dots)$ 
5   retorna ConsultaCaminho( $t, k, s, \dots$ )
```

2.3 Análise de Complexidade

Adota-se a seguinte notação: seja $|P_i|$ o número de vértices do polígono P_i , seja k o número de polígonos e $n = \sum_{i=1}^k |P_i|$ o número total de vértices.

Analisamos a complexidade de cada componente do algoritmo, de baixo para cima.

LocalizaPonto: A busca binária realiza $O(\log |P_i|)$ chamadas a **VerificaAresta**, cada uma em tempo $O(1)$. Portanto, **LocalizaPonto** tem complexidade $O(\log |P_i|)$.

ConsultaÚltimoPasso: No pior caso, o ponto consultado está na região de atravessar de todos os polígonos $P_i, P_{i-1}, \dots, P_1$, resultando em i chamadas a **LocalizaPonto**. A complexidade é portanto:

$$O\left(\sum_{j=1}^i \log |P_j|\right).$$

RegiãoDePrimeiroContato e MapaDeÚltimoPasso: Ambos iteram sobre os $|P_i|$ vértices de P_i , realizando uma chamada a **ConsultaÚltimoPasso** por vértice. A complexidade de cada um é:

$$O\left(|P_i| \sum_{j=1}^i \log |P_j|\right).$$

Algoritmo completo: O algoritmo calcula T_i e S_i para $i = 1, \dots, k$ e ao final consulta o k -path até t . O custo total é:

$$\begin{aligned} O\left(\sum_{i=1}^k |P_i| \sum_{j=1}^i \log |P_j|\right) &= O\left(\sum_{i=1}^k |P_i| \sum_{j=1}^k \log |P_j|\right) \\ &= O\left(n \sum_{j=1}^k \log |P_j|\right). \end{aligned}$$

A soma $\sum_{j=1}^k \log |P_j|$ é maximizada quando os polígonos têm o mesmo número de vértices, isto é, $|P_j| = n/k$ para todo j , o que dá $\sum_{j=1}^k \log(n/k) = k \log(n/k)$. Portanto, a complexidade total do algoritmo é:

$$O(nk \log(n/k)),$$

que coincide com a complexidade apresentada por Dror et al. [2].

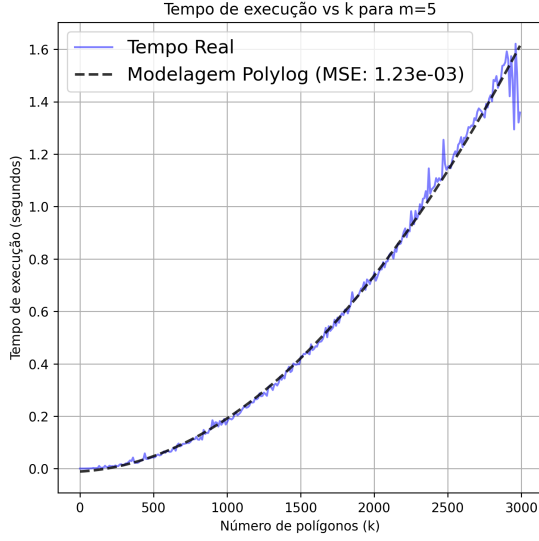
2.4 Experimentos Computacionais

Os experimentos foram conduzidos em um MacBook Pro com chip Apple M4 Pro (12 núcleos, sendo 8 de desempenho e 4 de eficiência) e 24 GB de memória. O código foi compilado com GCC 15.2.0 usando C++23 e otimização `-O3`. Para acelerar a execução dos testes, utilizamos paralelismo via OpenMP, porém os tempos registrados correspondem à execução sequencial de cada instância individualmente.

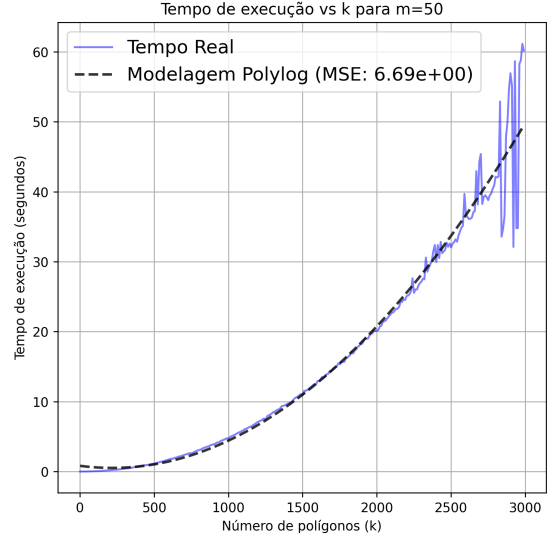
O objetivo dos experimentos é verificar que o comportamento assintótico do algoritmo está de acordo com a complexidade $O(nk \log(n/k))$ prevista pela análise teórica. Utilizamos instâncias sintéticas com k polígonos regulares de m vértices cada, dispostos de forma que todo ponto de P_i esteja na região de atravessar de P_j para todo $j < i$, representando o pior caso do algoritmo. Nessa configuração, $n = mk$ e a complexidade teórica é $O(m \log(m) \cdot k^2)$.

Realizamos dois experimentos. No primeiro, fixamos $m \in \{5, 50\}$ e variamos k de 1 até 3000, com 300 medições uniformemente espaçadas. Para cada valor de m , ajustamos um modelo da

forma $c_0 + c_1k + c_2k^2$ aos dados de tempo de execução por mínimos quadrados; a curva ajustada é exibida junto com os dados na Figura 9. Os resultados mostram que o tempo de execução cresce de forma quadrática em k , conforme previsto.

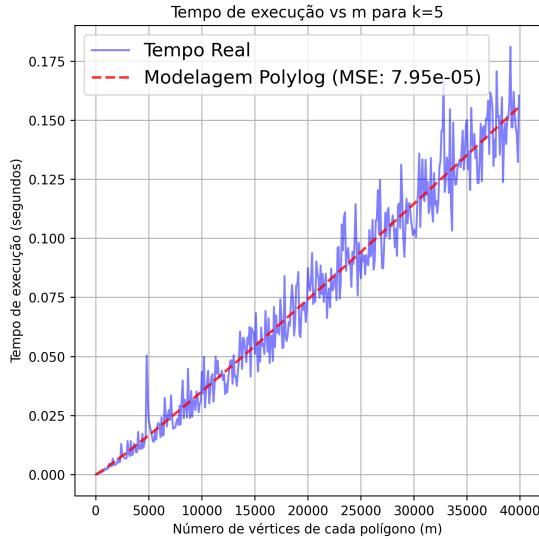


(a) Resultados para $m = 5$.

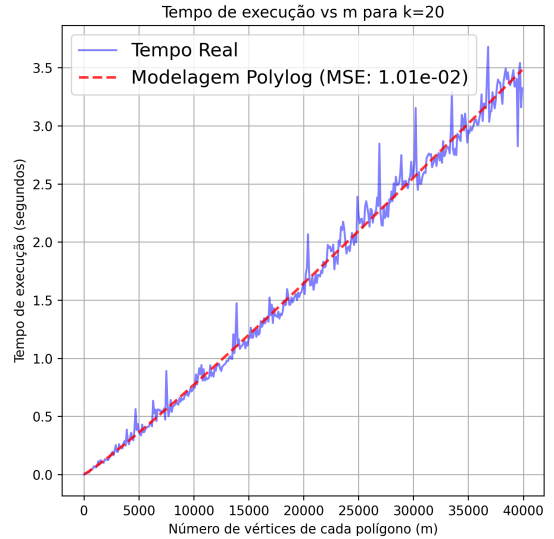


(b) Resultados para $m = 50$.

Figura 9: Tempo de execução em função de k para dois valores de m . A curva tracejada corresponde ao modelo ajustado por mínimos quadrados.



(a) Resultados para $k = 5$.



(b) Resultados para $k = 20$.

Figura 10: Tempo de execução em função de m para dois valores de k . A curva tracejada corresponde ao modelo ajustado por mínimos quadrados.

No segundo experimento, fixamos $k \in \{5, 20\}$ e variamos m de 3 até 40000, com 400 medições uniformemente espaçadas. Para cada valor de k , ajustamos um modelo da forma $c_0 + c_1(n + k) + c_2nk + c_3nk \log(n/k)$ por mínimos quadrados; a curva ajustada é exibida na Figura 10. As curvas têm aspecto aproximadamente linear, o que é esperado: o fator $\log(m)$ varia apenas de $\log(3) \approx 1,6$

a $\log(40000) \approx 10,6$ na faixa considerada, uma variação de apenas $6\times$ em uma escala de $40000\times$, tornando-o imperceptível visualmente. O ajuste do modelo polilogarítmico confirma, no entanto, que os coeficientes são consistentes com a complexidade teórica.

3 Ferramenta de Visualização

Como parte do trabalho desenvolvido neste período, implementamos uma ferramenta de visualização interativa para o TPP, disponível em <https://yushipy.github.io/PersonalPage/Visualizer/index.html>. A ferramenta permite construir instâncias do problema diretamente no navegador, definir os pontos s e t , editar polígonos e acionar o algoritmo para computar o caminho ótimo.

Para instâncias com polígonos convexos, é possível exibir o mapa de último passo do polígono selecionado, colorindo as regiões de vértice em vermelho, as regiões de aresta em verde e a região de atravessar em ciano. A Figura 11 apresenta uma captura de tela da ferramenta com uma instância de exemplo.

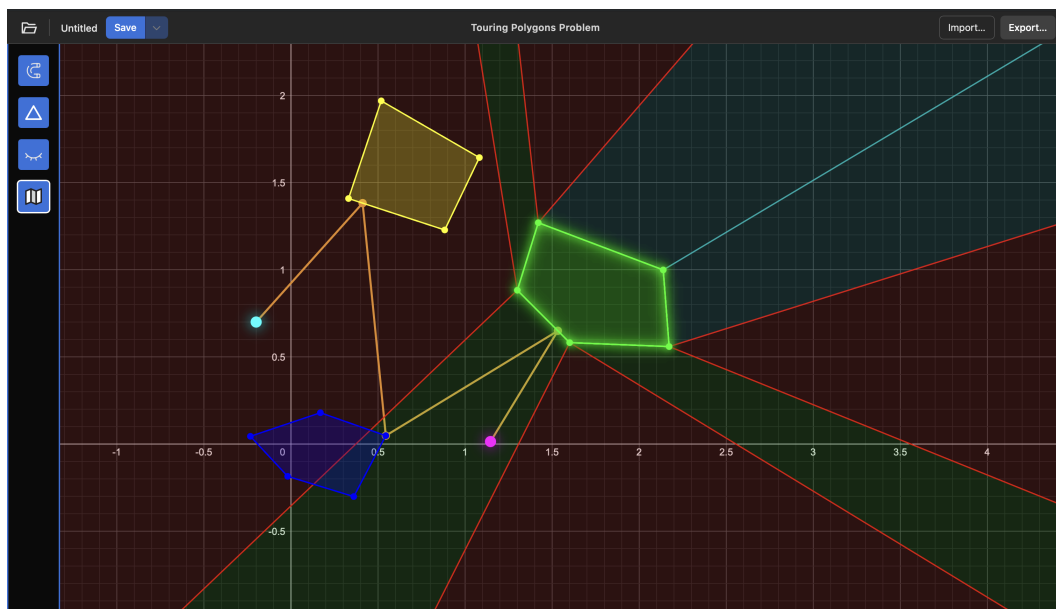


Figura 11: Ferramenta de visualização interativa do TPP, exibindo uma instância com três polígonos convexos, o caminho ótimo encontrado e o mapa de último passo S_3 do terceiro polígono.

A ferramenta foi projetada para acompanhar o desenvolvimento do projeto: à medida que implementarmos o suporte ao caso não convexo nas etapas seguintes, estenderemos a visualização para cobrir essas instâncias. No momento da escrita deste relatório, instâncias com polígonos não convexos desativam automaticamente a exibição dos mapas de último passo. Uma documentação detalhada das funcionalidades está disponível em <https://yushipy.github.io/PersonalPage/Visualizer/tpp-info.html#toolbar>.

4 Plano de Atividades para o Próximo Período

Dedicaremos o próximo período ao TPP não convexo com ordem fixa, que é NP-difícil [2]. A abordagem planejada consiste em decompor cada polígono não convexo em peças convexas, usando o algoritmo de Greene [3] disponível na biblioteca CGAL [5], e aplicar o algoritmo desenvolvido neste período a cada combinação possível de peças, explorando técnicas de Branch and Bound para tornar essa busca tratável na prática.

As atividades previstas são as seguintes:

1. Integrar a decomposição convexa de polígonos não convexos via algoritmo de Greene [3], disponível na biblioteca CGAL [5].
2. Formular e implementar o algoritmo de Branch and Bound para o TPP não convexo. Cada nó da árvore de busca atribui uma peça convexa a cada polígono; o limitante inferior é obtido substituindo os polígonos ainda não atribuídos pelos seus cascos convexas e resolvendo o TPP convexo resultante, o que mantém o cálculo do limitante em tempo polinomial em cada nó.
3. Calcular uma solução inicial de boa qualidade para inicializar o Branch and Bound, reduzindo o espaço de busca explorado.
4. Implementar um baseline via programação inteira mista com o solver Gurobi [4] para verificar a correte e comparar o desempenho do Branch and Bound.
5. Estender a ferramenta de visualização para suportar instâncias com polígonos não convexas.
6. Realizar experimentos computacionais para avaliar o desempenho do Branch and Bound em instâncias de diferentes tamanhos e características.

Paralelamente, submeteremos à FAPESP uma solicitação de bolsa BEPE para dar continuidade ao projeto em estágio de pesquisa no exterior. A proposta prevê uma visita de quatro meses à Université Laval, no Canadá, sob supervisão do Prof. Leandro Callegari Coelho, cujo grupo de pesquisa possui ampla experiência em métodos exatos para problemas de roteamento de veículos. O foco da etapa no exterior será o TPP sem ordem fixa de visita, variante NP-difícil que generaliza o TPP não convexo e resolve o problema de roteamento de veículo único com vizinhanças poligonais não convexas.

Referências

- [1] E. M. Arkin and R. Hassin. Approximation algorithms for the geometric covering salesman problem. *Discrete Applied Mathematics*, 55(3):197–218, 1994.
- [2] M. Dror, A. Efrat, A. Lubiw, and J. S. B. Mitchell. Touring a sequence of polygons. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '03, pages 473–482, 2003.

- [3] D. H. Greene. The decomposition of polygons into convex parts. In F. P. Preparata, editor, *Computational Geometry*, volume 1 of *Advances in Computing Research*, pages 235–259. JAI Press, London, England, 1983.
- [4] Gurobi Optimization LLC. Gurobi optimizer 12.0, 2025.
- [5] The CGAL Project. CGAL 6.0, 2025.